SHORT REPORT — AI RESUME SCREENING SYSTEM

1. PROBLEM

Recruiters and hiring teams routinely receive far more resumes than they can carefully read. Manual screening is slow, inconsistent between reviewers, and gives candidates no visibility into why they were or weren't shortlisted.

Goal: given a job description and a set of resumes, automatically score and rank candidates, and make the reasoning behind each score visible and explainable to the user — not just a black-box number.

Input: a job description (text) plus a set of resumes (sample data, manual entry, or uploaded PDFs).

Output: a ranked list of candidates with scores, tiers (A–D), matched and missing skills, and a plain-language explanation for each.

Constraints: must run without external paid APIs, must degrade gracefully if optional ML/NLP libraries aren't installed, and must show its work (explainability and evaluation), not just a final answer.

2. METHOD / ARCHITECTURE

The app is a single Flask application (main.py) organised into the modular structure required by the project guide:

A) Problem Setup — job-description input/preset selector, sample vs. manual vs. PDF-upload candidate input, weight sliders, and input validation with inline error messages.

B) Core Logic — load_data, preprocess_data, run_screening (the run_model_or_algorithm equivalent), generate_explanation, and the chart-data builders that feed the UI.

C) Visual UI — bar, pie, line, scatter, radar, and heatmap charts; a skill network graph; a score grid; and a step-by-step timeline — plus sliders, toggles, dropdowns, a result panel, and status messages.

D) Explainability — per-candidate verdicts and factor breakdowns, a forward-chaining rule trace, and an NLP chatbot panel that answers plain-English questions about the results.

E) Evaluation — precision/recall/F1, runtime and peak memory, and multiple approach-comparison panels (TF-IDF vs Semantic; Rule-Based vs TF-IDF vs Full AI; and BFS/DFS/UCS/Greedy/A*/Hill Climbing).

3. AI COMPONENTS USED (SECTION 5 OPTIONS)

Option 1 — Rule-based AI (forward chaining)
Facts are derived per candidate (high_score, meets_base_requirement, has_required_education, etc.) and IF–THEN rules fire in sequence to reach a final recommendation (RECOMMEND FOR INTERVIEW / CONSIDER WITH CAUTION / NOT RECOMMENDED). Every fired rule and its explanation is shown to the user.

Option 2 — Search / Optimisation AI
Selecting the best K-candidate shortlist (the one maximising total combined score) is framed as a state-space search over "include / skip" decisions for each candidate — structurally identical to 0/1-knapsack. Six algorithms solve the same problem so their trade-offs can be compared directly:

BFS (Breadth-First Search)
Strategy: Explores level-by-level
Optimality: Optimal, but explores the most nodes

DFS (Depth-First Search)
Strategy: Explores depth-first with backtracking
Optimality: Not guaranteed optimal

UCS (Uniform-Cost Search)
Strategy: Expands lowest-cost node first (cost = -score)
Optimality: Optimal

Greedy Best-First Search
Strategy: Expands by heuristic estimate only
Optimality: Not guaranteed optimal

A* Search
Strategy: Cost + admissible heuristic (best-case remaining score)
Optimality: Optimal, fewest nodes expanded

Hill Climbing (Local Search)
Strategy: Starts from one solution, swaps in better candidates
Optimality: Fast, but can settle on a local optimum

For each run the app reports nodes expanded, max frontier size, runtime, and the first states explored, so the difference between "search the whole

tree" (BFS/DFS) and "prune with a heuristic" (A*) is visible rather than just asserted.

Option 3 — Machine Learning AI
Two layers: (a) the core screening score uses TF-IDF vectorisation plus cosine similarity, optionally boosted with a pretrained Sentence-Transformer (all-MiniLM-L6-v2) for semantic similarity; (b) a dedicated ML Training panel trains three real scikit-learn models on the screened candidates so the guide's "train a model / show training results and predictions visually" requirement is met literally, not just implied by the similarity score:

Classification — Logistic Regression
Trained on: TF-IDF, Skill, Experience
Predicts: Qualified / Not-Qualified
Reported metrics: Accuracy, Precision, Recall, F1, confusion matrix

Regression — Linear Regression
Trained on: Skill, Experience
Predicts: Final weighted score
Reported metrics: R-squared, MAE, RMSE, residuals

Clustering — K-Means + PCA
Trained on: TF-IDF, Skill, Experience, Education (unsupervised)
Predicts: Candidate group ("High / Moderate / Low Fit")
Reported metrics: Silhouette score, % variance explained

Each task deliberately withholds part of the feature set from its target so predictions are genuinely learned, not a restatement of the scoring formula. With the small candidate counts typical of a resume-screening demo, a single train/test split is unreliable, so classification and regression automatically switch to Leave-One-Out cross-validation below 8 samples — and the UI states which evaluation method was used rather than hiding it.

Option 4 — NLP / LLM-assisted AI
A full NLP pipeline (tokenisation, stop-word removal, lemmatisation, POS tagging, and Named Entity Recognition via spaCy, with regex fallbacks if spaCy isn't installed) is run on demand and shown step-by-step, alongside two additional standard NLP features: bigram extraction (surfaces multi-word phrases like "machine learning" that single-keyword frequency misses) and lexicon-based sentiment/tone analysis (flags resume/JD language as Positive/Neutral/Negative based on a professional-context word list — e.g. "strong", "proficient" vs. "lack", "limited"). An extractive summariser (frequency-scored sentence selection) produces a plain-language summary of any resume or job description. A rule-driven chatbot panel

answers canned but data-grounded questions ("Who is the best candidate and why?", "What skills are most missing?") by reading the live screening results — no external LLM API is called, so there are no external cost/privacy concerns to disclose.

4. RESULTS

On the bundled 12-candidate sample dataset against a "Senior Python Developer" job description:

- TF-IDF-only scoring and the semantic-enabled scoring agree on the top 2 candidates but re-order the middle of the pack — demonstrating that keyword overlap and meaning-based similarity capture different signals.

- The Rule-Based approach is stricter than the Full AI approach: it qualifies fewer candidates because it does not credit partial skill overlap the way the weighted ML score does.

- For shortlist search (K = 4), UCS and A* both find the true optimum while expanding only about 5 nodes, versus about 2,400 nodes for BFS/DFS on the same 12-candidate set — roughly a 480x difference. Greedy and Hill Climbing are faster still but occasionally land on a slightly lower-scoring shortlist, illustrating the classic completeness vs. speed trade-off in search/optimisation AI.

- With 12 candidates, the trained classifier now uses a genuine 70/30 train-test holdout split (rather than falling back to cross-validation), giving a more realistic accuracy estimate; the regression model explained about 96% of score variance (R-squared = 0.96) from just skill and experience, and K-Means consistently separated candidates into a clear "High Fit" cluster from the rest (silhouette score = 0.39), confirmed visually by the PCA scatter plot.

- The evaluation panel reports runtime in single-digit milliseconds for all screening approaches on this dataset size, so the system remains usable interactively even with every module enabled.

Screenshots of each module in action are in the screenshots folder of the submission.

5. LIMITATIONS AND FUTURE IMPROVEMENTS

- Skill and education extraction use keyword/regex matching, not full resume-parsing NLP — a mis-formatted resume can under-report skills.

- The Sentence-Transformer model and spaCy model are optional; scoring quality is lower when they aren't installed (TF-IDF-only fallback).

- The search/optimisation module optimises total score only; it doesn't yet factor in diversity or role-fit balance across the shortlist.

- No external LLM API is used for the chatbot — responses are template-based over the real results rather than free-form generation. A future version could add an optional LLM-backed chat mode, with its API/prompt/output and privacy implications disclosed as required by Section 5.